

```

// List.h begins here-----
#ifndef LIST_H
#define LIST_H
#include <iostream>
using namespace std;
typedef int datatype;

class list {
    class Node{
    public:
        datatype value;
        Node *next;
        Node():next(0){}
        Node(datatype val):value(val),next(0){}
    } ;

    int mySize;
    Node *first;
public:

    list(); // constructor
    ~list(); // destructor
    list(list &); // copy constructor
    void insert(int , datatype &); // insert the value at ith position
    void erase(int ); // erase the item indexed by the argument
    // some helping functions
    Node * Index2Pointer(int );
    bool empty(); // checks if list is empty
    int returnSize(); // returns the current size of the list
    datatype readithValue(int); // read the ith value where is is passed as
    argument
    // Overloaded operator
    list & operator=(list &); // overloaded assignment operator
    friend ostream & operator<< ( ostream & , list &);
};
#endif
// List.h ends here-----

```

```

// List.cpp begins here-----
#include <iostream>
#include "List.h"
using namespace std;
//Constructor
list::list():first(0),mySize(0){}

// Destructor to destroy the list
list::~~list(){
    list::Node *temp1, *temp2; // list:: indicates that Node datatype is in
the scope of list
    temp1 = first;
    while(temp1 != 0){
        temp2 = temp1->next;
        delete temp1;
        temp1 = temp2;
    }
}
list::list(list &Original){
    first = new Node(Original.first->value);
    Node *duplicate = first;
    Node *currnode= Original.first->next;
    while(currnode != 0){
        duplicate->next = new Node(currnode->value);
        currnode = currnode->next;
        duplicate = duplicate->next;
    }
}

// helper function , Index2pointer returns corresponding address of node
containing the address of indexed element
// first-->(value, address1)-->(value,address2).....-->(value,addressN)---->
NULL
// in above example, index 0 refers to value of first because first addresses
index 0 element, index 2 refers to node that contains address2,
// in this case address of node containing (value,address2) pair.
list::Node * list::Index2Pointer(int index){ // return type should be
specified as (list::Node *) because we have to tell compiler that datatype
Node is in list
    Node *ptr = first;
    //
otherwise, it would not recognize node, recall that it is hidden from outside
of list.
    for (int i = 1; i < index; i++){
        ptr = ptr->next;
    }
    return ptr;
}

bool list::empty(){return (mySize == 0);} // empty function

int list::returnSize(){return mySize;} // returns current size

datatype list::readithValue(int index){ // return the index-th item from the
list
    if (index < 0 || index > mySize){
        cerr<< "Illegal location "<< endl;
    }
}

```

```

        exit(1);
    }
    if (index == 0)
        return first->value;
    else {
        Node *location = Index2Pointer(index); // corresponding pointer is
returned
        return location->next->value;
    }
}
// insert function
void list::insert(int index, datatype & num){
    if (index < 0 || index > mySize){
        cerr<< "Illegal location "<< endl;
        return;
    }
    mySize++;
    Node *newNode = new Node(num);
    if (index == 0){
        newNode->next=first;
        first = newNode;
    }
    else{
        Node *location = Index2Pointer(index); // corresponding pointer is
returned
        newNode->next = location->next; // setting the new node pointer to the
address pointed by next element of node pointed by pointer location
        location->next = newNode; // this completes the join by making next
element of node pointed by location point to newNode.
    }
}

ostream & operator<<( ostream & out, list &aList){
    list::Node *ptr = aList.first;
    while(ptr != 0){
        out << ptr->value <<" ";
        ptr = ptr->next;
    }
    out<<endl;
    return out;
}

void list::erase(int index){ // erase the index-th item from the list
    if (index < 0 || index > mySize){
        cerr<< "Illegal location "<< endl;
        return;
    }
    mySize--;
    list::Node *ptr;
    if (index == 0){
        ptr = first;
        first = first->next;
        delete ptr;
    }
}

```

```
        else{
            list::Node *temp;
            ptr = Index2Pointer(index);
            temp = ptr->next;
            ptr->next=ptr->next->next;
            delete temp;
        }
    }

list & list::operator=(list &right){

    if (this == &right)        return *this;

    else{
        this->~list();
        list::list(right);
    }
    return *this;
}
// List.cpp  ends here-----
```